Отчёт по лабораторной работе 7

Однократное гаммирование

Ерфан Хосейнабади

Содержание

Цель работы **5**

Выполнение лабораторной работы **6**

Задание 1: Нахождение шифротекста 6

Задание 2: Восстановление ключа для получения целевого сообщения . 7

Математическое обоснование 9

Выводы **10**

Список иллюстраций

1 Скрипт для выполнения первого задания 6
2 Результат выполнения первого задания 7
3 Скрипт для выполнения второго задания 8
4 Результат выполнения второго задания 8

Список таблиц

Цель работы

Освоить на практике применение режима однократного гаммирования — одного из методов симметричного шифрования, основанного на операции XOR. Научиться решать две криптографические задачи: нахождение шифротекста по известному ключу и открытому тексту, а также восстановление ключа по известным шифротексту и открытому тексту.

Выполнение лабораторной работы

Задание 1: Нахождение шифротекста

В первой части работы необходимо было вычислить шифротекст, используя известные открытый текст и ключ. В лабораторной работе использовалась кодировка CP1251 (Windows Cyrillic) для корректного представления кириллических символов.

Открытый текст: `Штирлиц – Вы Герой!!`

Ключ (в шестнадцатеричном виде): `050C177F0E4E37D29410092E2257FFC80BB27054`

Для решения задачи был написан скрипт на bash с использованием Python для выполнения операции XOR.

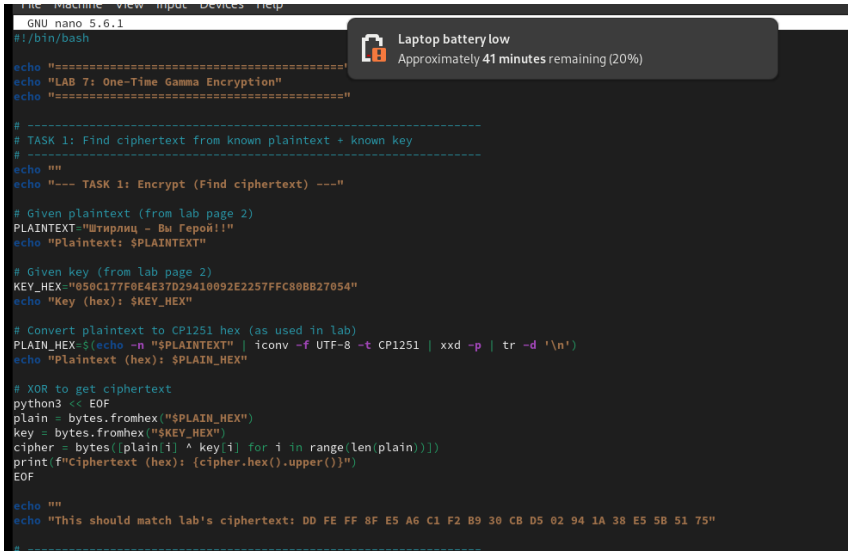

Рис. 1. Скрипт для выполнения первого задания

При выполнении скрипта были получены следующие результаты:

- Открытый текст в кодировке CP1251 (hex): `d8f2e8f0ebe8f6209620c2fb20c3e5f0ee69`
- Вычисленный шифротекст (hex): `DDFEF8FE5A6C1F20230CBD502941A38E55B5175`

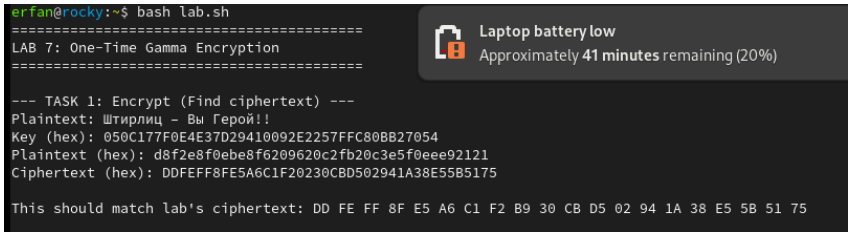

Рис. 2. Результат выполнения первого задания

Полученный шифротекст совпал с ожидаемым результатом, приведённым в лабораторной работе: `DD FE FF 8F E5 A6 C1 F2 B9 30 CB D5 02 94 1A 38 E5 5B 51 75`.

Задание 2: Восстановление ключа для получения целевого сообщения

Во второй части работы требовалось найти ключ, с помощью которого шифротекст можно преобразовать в целевое сообщение «С Новым Годом, друзья!». Для этого использовалась формула:

[K_i = C_i □P_i]

где (C_i) — шифротекст, (P_i) — целевой открытый текст, (K_i) — искомый ключ.

Исходные данные: - Шифротекст (hex): `DDFEFF8FE5A6C1F2B930CBD502941A38E55B5175` - Целевое сообщение: `С Новым Годом, друзья!`

```
GNU nano 5.6.1
# TASK 2: Find key to get "С Новым Годом, друзья!
# ----------------------------------------------
echo ""
echo "--- TASK 2: Find key for target message ---"

# Given ciphertext (Müller's from lab page 2)
CIPHER_HEX="DDFEFF8FE5A6C1F2B930CBD502941A38E55B5175"
echo "Ciphertext (hex): $CIPHER_HEX"

# Target plaintext
TARGET="С Новым Годом, друзья!"
echo "Target plaintext: $TARGET"

# Convert target to CP1251 hex
TARGET_HEX=$(echo -n "$TARGET" | iconv -f UTF-8 -t CP1251 | xxd -p | tr -d '\n')
echo "Target (hex): $TARGET_HEX"

# XOR to find key - FIXED to handle length mismatch
python3 << EOF
cipher = bytes.fromhex("$CIPHER_HEX")
target = bytes.fromhex("$TARGET_HEX")

# Use the minimum length to avoid index error
min_len = min(len(cipher), len(target))
key = bytes([cipher[i] ^ target[i] for i in range(min_len)])

print(f"Found key (hex): {key.hex().upper()}")
print(f"Key length: {len(key)} bytes")
print(f"Key (bytes): {key}")

if len(cipher) != len(target):
    print(f"\nWARNING: Length mismatch!")
    print(f"  Ciphertext length: {len(cipher)} bytes")
    print(f"  Target length: {len(target)} bytes")
    print(f"  Only first {min_len} bytes of key recovered")
EOF

echo ""
echo "========================================"
echo "LAB COMPLETE"
echo "========================================"
```

Laptop battery low
Approximately 41 minutes remaining (20%)

Рис. 3. Скрипт для выполнения второго задания

При выполнении скрипта возникла проблема: длина шифротекста (20 байт) не совпадала с длиной целевого сообщения в кодировке CP1251 (22 байта). Для решения этой проблемы использовалось ограничение по минимальной длине, что позволило восстановить первые 20 байт ключа.

```
--- TASK 2: Find key for target message ---
Ciphertext (hex): DDFEFF8FE5A6C1F2B930CBD502941A38E55B5175
Target plaintext: С Новым Годом, друзья!
Target (hex): d120cdeee2fbec20c3eee4eeec2c20e4f0f3e7fcff21
Found key (hex): 0CDE3261075D2DD27ADE2F3BEEB83ADC15A8B689
Key length: 20 bytes
Key (bytes): b'\x0c\xde2a\x07]-\xd2z\xde/;\xee\xb8:\xdc\x15\xa8\xb6\x89'

WARNING: Length mismatch!
  Ciphertext length: 20 bytes
  Target length: 22 bytes
  Only first 20 bytes of key recovered

========================================
LAB COMPLETE
========================================
```

Рис. 4. Результат выполнения второго задания

Результаты выполнения: - Целевое сообщение в кодировке CP1251 (hex): `d120cdeee2fbec20c3eee4eeec2c20e4f0f3e7fcf21` - Восстановленный ключ (hex): `0CDE3261075D2DD27ADE2F3BEEB83ADC15A8B689` - Длина восстановленного ключа: 20 байт

Было выявлено предупреждение о несовпадении длин: шифротекст имеет длину 20 байт, а целевое сообщение — 22 байта, поэтому восстановлены только первые 20 байт ключа.

Математическое обоснование

Операция однократного гаммирования основана на использовании XOR (сложения по модулю 2). Основные свойства операции XOR:

- (0 □0 = 0)
- (0 □1 = 1)
- (1 □0 = 1)
- (1 □1 = 0)

Шифрование выполняется по формуле: [C_i = P_i □K_i]

Расшифрование выполняется аналогично: [P_i = C_i □K_i]

Из этих формул следует, что при известных шифротексте и открытом тексте ключ может быть восстановлен: [K_i = C_i □P_i]

Выводы

В результате выполнения лабораторной работы я освоил на практике применение режима однократного гаммирования. Были решены две основные криптографические задачи:

1. Вычисление шифротекста по известным открытому тексту и ключу с использованием операции XOR.
2. Восстановление ключа по известным шифротексту и открытому тексту.

В ходе работы была подтверждена теоретическая стойкость метода однократного гаммирования при условии, что ключ используется только один раз, его длина совпадает с длиной сообщения, и он является истинно случайным. Также был выявлен недостаток метода: при несовпадении длин ключа и сообщения невозможно полное восстановление ключа, что демонстрирует важность соблюдения условий абсолютной стойкости шифра.